

USB Drivers in Linux



This article, which is part of the series on Linux device drivers, gets you started with writing your first USB driver in Linux.

Part—11

Pugs and Shweta decided to use a pen drive to explore the world of USB drivers in Linux. Pugs' preferred to pick up a USB device, and write a driver for it, to experiment with. So they chose the pen drive (a.k.a. USB stick) that was at hand—a JetFlash from Transcend, with vendor ID 0x058f and product ID 0x6387.

USB device detection in Linux

Whether or not there is a Linux driver for a USB device, a valid USB device will always be detected at the hardware and kernel spaces of a USB-enabled Linux system, since it is designed (and detected) as per the USB protocol specifications. Hardware-space detection is done by the USB host controller—typically a native bus device, like a PCI device on x86 systems. The corresponding host controller driver would pick and translate the low-level physical layer information into higher-level USB protocol-specific information. The USB protocol formatted information about the USB device is then populated into the generic USB core layer (the *usbcore* driver) in kernel-space, thus enabling the detection of a USB device in kernel-space, even without having its specific driver.

After this, it is up to various drivers, interfaces and applications (which are dependent on the various Linux distributions), to have the user-space view of the detected devices. Figure 1 shows a top-to-bottom view of the USB subsystem in Linux. A basic listing of all detected USB devices can be obtained using the *lsusb* command, as root.

Figure 2 shows this, with and without the pen drive plugged in. A *-v* option to *lsusb* provides detailed information. In many Linux distributions like Mandriva and Fedora, the *usbfs* driver is loaded as part of the default configuration. This enables the detected USB device details to be viewed in a more techno-friendly way through the */proc* window, using *cat /proc/bus/usb/devices*. Figure 3 shows a typical snippet of this, clipped around the pen drive-specific section. The listing basically contains one such section for each valid USB device detected on the system.

Decoding a USB device section

To further decode these sections, a valid USB device needs to be understood first. All valid USB devices contain one or more configurations. A configuration of a USB device is like a profile, where the default one is the most commonly used. As such, Linux supports only one configuration per device—the default one. For every configuration, the device may have one or more interfaces. An interface corresponds to a function provided by the device. There would be as many interfaces as the number of independent functions provided by the device. So, if for instance, an MFD (multi-function device) USB printer can do printing, scanning and faxing, then it most likely would have at least three interfaces, one for each of the functions. So, unlike other device drivers, a USB device driver is typically associated/written per interface, rather than the device as a whole—which means that one USB device may have multiple device drivers, and different device